

# SQL Injection Detection and Prevention Techniques

1<sup>st</sup> Sarita Sangrez

Department of Cyber Security  
FAST NUCES Islamabad Campus  
Islamabad, Pakistan  
i232088@isb.nu.edu.pk

2<sup>nd</sup> Amna Ali

Department of Cyber Security  
FAST NUCES Islamabad Campus  
Islamabad, Pakistan  
i232067@isb.nu.edu.pk

3<sup>rd</sup> Eman Akbar

Department of Cyber Security  
FAST NUCES Islamabad Campus  
Islamabad, Pakistan  
i221588@nu.edu.pk

**Abstract**—Structured Query Language (SQL) injection is one of the most significant cybersecurity threats due to its frequent use in real-world attacks. Although common prevention methods such as input validation, parameterized queries, and firewalls provide some protection, many fail against advanced SQL injection techniques. To address this issue, this case study introduces a machine learning-based detection framework using a random forest ensemble model for accurate and interpretable SQL injection identification. The approach utilizes a dataset containing both genuine and malicious SQL queries. Data pre-processing is performed using a query-cleaning function, and feature selection incorporates TF-IDF vectorization along with handcrafted features to capture query patterns. The trained random forest model is integrated with SHAP explainability tools, a GUI for real-time detection, and a logging module for storing prediction results. The proposed framework also demonstrates safe execution practices to further reduce the risk of SQL injection attacks.

**Index Terms**—SQL Injection, Machine Learning, Random Forest, Cybersecurity

## I. INTRODUCTION

SQLi is a significant web security issue that has always been popular in the OWASP Top Ten [1]. It uses incorrect input verification, allowing attackers to manipulate SQL queries and retrieve sensitive information. Conventional firewall systems such as sanitization, prepared statements, and WAFs are unable to counter advanced, obfuscated attacks.

In order to fill these gaps, recent studies have looked at machine learning (ML) and deep learning (DL) tools. Semantic models like synBERT [2] are more effective at generalization by comprehending query intent, whereas feature-engineered deep models have been shown to have high accuracy [3] but are unintelligible. Random Forest and boosting models are also ensemble techniques providing some performance–transparency trade-off, particularly when accompanied by explainability systems such as SHAP and LIME [4].

The present study is based on such developments by assessing the performance of SQLi detection using a Kaggle dataset, with special focus on the Random Forest model since it is proven to have high accuracy, robustness, and interpretability. This is aimed at developing a usable and justifiable SQL injection detection framework that can be deployed in the real world. Accordingly, we investigate the following research question: **RQ1: Can a Random Forest-based SQL injection detection system, trained on a real-world SQL injection**

**dataset and enhanced with SHAP explanations and secure query execution, achieve near-perfect detection performance (AUC  $\geq$  0.99) while remaining interpretable enough for practical deployment?**

## II. LITERATURE REVIEW

### A. Enhancing Structured Query Language Injection Detection with Trustworthy Ensemble Learning and Boosting Models Using Local Explanation Techniques

Hussain et al. [4] proposed a detailed study for SQL injection detection that uses ensemble and boosting-based machine learning models, some of them being Decision Tree, Random Forest, XGBoost, AdaBoost, Gradient Boosting Decision Trees (GBDT), and Histogram Gradient Boosting Decision Trees (HGBDT). Their study tackled the problem of limited model transparency, which is a constraint in traditional SQL injection defence methods. This was done by incorporating explainable AI (XAI) techniques, specifically SHAP (Shapley Additive Explanations) and LIME (Local Interpretable Model-Agnostic Explanations), which help to interpret and visualize model decision processes. Random Forest showed high accuracy among the models tested as it achieved high detection performance and maintained its interpretability. This work emphasizes on both accuracy and trustworthiness which is important because unlike the previous studies that rely only on performance, this study recognizes the importance of explainability in ML based security systems. The framework includes SHAP and LIME that allows analysts to understand which features mostly affect model predictions [5], [6]. The study provides a basis for researchers and practitioners who look for both high performing and interpretable SQL injection detection systems, since it evaluates multiple ensemble and boosting algorithms under controlled experimental conditions. But several limitations remain despite these contributions. This technique was used on a dataset that was a bit restricted and might not fully represent the variable real world SQL injection attacks. So, this creates an uncertainty about how the models perform on large or obfuscated datasets. Also, XAI techniques improved interpretability, but the study mostly focused on detection and didn't address how to automate prevention or mitigation in detail. These limitations provide us with opportunities for further practical validation. This present study builds on Hussain et al.'s findings by evaluating the Random Forest model on a publicly available SQL injection

dataset from Kaggle, which provides good evidence of its effectiveness in diverse and realistic data conditions. By doing this, the work not only confirms the model's high accuracy but also assesses if it's suitable for deployment in today's web security environments, which connects theoretical performance to practical applications.

### *B. SQL Injection Vulnerability Detection Using Deep Learning: A Feature-based Approach*

Hassan *et al.* [7] proposed a framework for effective detection of SQL injection (SQLi) vulnerabilities in web applications using deep learning techniques, a feature-oriented approach combined with neural networks. This approach was used to improve detection accuracy compared to other machine learning models such as Support Vector Machines (SVM) and Random Forest. To implement this, the authors constructed a custom dataset consisting of 1,850 web application samples, each with 19 SQLi related features such as input validation, exception handling, illegal input acceptance, and database information disclosure described in Hassan *et al.* [7], nineteen key features were selected for SQLi detection, each defined by a specific logical rule [7]. The study applied feature ranking methods, specifically correlation coefficient analysis and chi-square tests, to refine the most influential attributes and improve the learning process.

A hybrid approach combining feature extraction, preprocessing, feature selection and deep neural network classification was taken. The neural network model is utilized to predict the SQLi vulnerability of the web with the dataset. Logical and security-based feature extraction, followed by preprocessing to handle inconsistencies, missing values, and data standardization. Feature selection using correlation and chi-square to optimize the feature set were chosen because of model independency. A feed-forward neural network with rectified linear unit (ReLU) in the first layer and sigmoid activations in the second layer. During the training process, network model is tuned to minimize the value of the loss function. The neural network is modelled with dropout regularization (rate 0.2) and K-fold cross-validation to avoid overfitting.

The model achieved a classification accuracy of 98.04%, exceeding the accuracy rates of SVM (94.66%) and Random Forest (97.33%). This demonstrates that deep learning can capture complex nonlinear relationships between web features and vulnerabilities more effectively than traditional algorithms. The primary part of the paper is the deep learning part in SQL injection vulnerability prediction. Further the paper states that future focus should be on building an automated tool and an effective way to prevent SQL injection.

However, several limitations were acknowledged. The dataset was manually constructed without real-time web traffic validation and limited to specific features. The focus was on SQLi and not other possible web attacks. Moreover, as the research was on static feature analysis, larger datasets might need more improvements and processing power which was limited for static analysis. Hence, the scalability was not evaluated. Overall the findings successfully demonstrated

that deep learning like neural networks along with effective features and engineering can achieve superior accuracy in detecting SQLi vulnerabilities

### *C. A Semantic Learning-Based SQL Injection Attack Detection Technology (synBERT) (2023)*

SQL Injection (SQLi) has been one of the most common attacks to the web-based systems, which allows the attackers to harness the database queries and steal the sensitive information [1]. Initial studies by Halfond *et al.* [8] gave a conceptual basis on the detection strategies by first presenting a basic categorization of SQLi attacks and the corresponding countermeasures. The S-QL self-learning system, that is likewise proposed by Valeur *et al.* [9], is an automatic system that learns query patterns that amount to anomalies and optimizes the model with time—a important change to a more adaptive protection as opposed to a less adaptive one. Equally, Wiese *et al.* [10] used dynamic taint analysis to monitor untrusted input in program execution resulting in better runtime prevention than with the use of static filters.

Huang *et al.* [11] used a combination of both the static code analysis and dynamic testing to determine vulnerabilities of the web applications at the application layer to enhance their prevention during deployment. Subsequent literature incorporated machine learning and data mining solution to SQLi detection. Sun *et al.* [12] presented the idea of using ensemble-learning on the basis of feature representation of a query token, which is involved in the identification of an abnormal behavior, whereas Lee *et al.* [13] manifested the success of deep neural networks to differentiate attack payloads with great accuracy. Khare *et al.* [14] built upon these results with hybrid architecture of rule-based filtering and supervised models, which enhanced the control of false-positives as well as flexibility to unknown attacks.

In addition to injection specific defenses, encryption and preservation of privacy has become the subject of research on database security. Agrawal *et al.* [15] proposed an idea of order-preserving encryption (OPE), which enables a range query of the encrypted numeric data and preserves the confidentiality of the information data. Popa *et al.* [16] came up with CryptDB, a useful system that permits the SELECTing of SQL queries by encrypted databases through the application of variable encryption layers depending on the type of query. The tradeoff between query Politics and these papers is mentioned in cryptographic Power. In accordance with the security implications that are, more strictly speaking, deterministic and order-preserving schemes, according to Boldyreva *et al.* [17] and Subashini and Kavitha [18], as well. Devoted to the question of leakage and the need to handle it, cloud environments have keys in layers. Recent surveys and comparative studies, such as those found by Bertino and Sandhu [19] and Zhang *et al.* [20], to grant complete protection, intrusion prevention, runtime query analysis, and encryption-based data. This must be paired with confidentiality. Taken together, the rule-based approach has proven to be a trend in the literature reviewed, with prevention yielding to intelligent and adaptive models.

Combined with privacy-assuring encryption, which highlights the significance of multi-layered designs, it not only detects threats in new web systems effectively but also guarantees the security of data.

### III. METHODOLOGY / PROPOSED FRAMEWORK / SOLUTION

In the current study, a machine learning-based approach is used. SQLi attack detection and prevention framework. The method uses the Random Forest classifier that relies on the random forest. Has high accuracy, robustness, and the capacity to. Process both textual and numerical hybrid data. This design is motivated by the previous ensemble-based models, including those. Defined in [4], in which it is possible to combine several decision trees. Higher reliability of detection and less overfitting. The system proposed is in the form of a holistic structure. That incorporates a machine learning detection model, a secure, enforced logging system, and a prevention demonstration. With parameterized queries. Hence, the complete system represents detection and prevention goals.

#### A. Dataset Description

The publicly available dataset is the one that is used to train the model. Data on SQL Injection respectively, received on Kaggle. *SQL Injection Dataset* benign and malicious SQL queries. Each record in the dataset is composed of different attributes that characterize the query. composition, including [21].

- Query text
- Query length
- Occurrence of SQL keywords
- Presence of special characters
- Ratio of digits and operators

The following is necessary to achieve a balanced and reliable model learning, pre-processing procedures were done:

- 1) Null and duplicate values are eliminated.
- 2) Text normalization through to lowercase conversion, elimination of comments punctuation, and tokenization.
- 3) Mining of numerical attributes like ratio of digits and keyword density.
- 4) Separating the data into 80% for training and 20% for testing to guarantee generalization.

The suggested framework is multi-stage. That is a combination of text vectorization, feature engineering, ensemble, learning, elucidation, and safeguarding recording. The key stages are as follows:

#### B. Proposed Framework

The proposed framework follows a multi-stage workflow that combines text vectorization, feature engineering, ensemble learning, explainability, and secure logging. The key stages are as follows:

- 1) Data cleaning: The raw SQL queries are purged by the `cleanquery()` function which eliminates unnecessary whitespaces, transforms all the text to lower case, and

normalizes characters. This is done to provide homogeneous and consistent text prior to feature extraction.

- 2) Extraction of features: This model employs a hybrid representation of features which is a combination of TF-IDF vectorization and hand crafted features. TF-IDF identifies patterns and contextual hints of SQL queries, including patterns, like `OR 1=1` or `UNION SELECT`. The extracted features, which are handcrafted and obtained with `extracthandcraftedfeatures`, are query length, frequency of operator, frequency of digit, ratio of digit and frequency of frequency of keyword. The combination of these features creates a single feature space which contains the structural and semantic properties of the query.
- 3) Model training: A hybrid features classifier is trained on the hybrid features. The trained model is stored as `rfmodel.pkl` and classifies SQL queries as either: Normal (0), i.e. a legitimate SQL query, or Attack (1) i.e. a potential SQL injection attempt. The trained model is saved in the `artifacts/` directory as the TF-IDF vectorizer and used to deploy and use the GUI.
- 4) Explainability layer (SHAP): SHAP explainers are incorporated to give explanation on the predictions made by the model. They assist in understanding the extent to which various characteristics can be used to classify the outcome and which of the patterns can be recognized as the most appropriate to describe the behavior of SQL injection.
- 5) GUI and prediction: The `guittk.py` is a graphical interface based on `tkinter` which allows real time testing. Users are able to enter any SQL query, observe the predicted output of the model (normal or attack) and the confidence score. This interface enables the model to be integrated in the real world testing.
- 6) Prevention and secure logging: The `securelog.py` module encrypts all the predictions with AES (Fernet) and saves them in `detections.log.enc`, which ensures the privacy, traceability, and integrity of the logged results. The `preventiontechniques.py` script illustrates safe and unsafe executions of SQL queries which illustrates how parameterized queries can be used to avoid SQL injection despite providing malicious inputs.

#### C. Tools and Environment

TABLE I: Tools and Environment Used

| Component            | Tool / Library                     |
|----------------------|------------------------------------|
| Programming Language | Python 3.12                        |
| GUI Framework        | Tkinter                            |
| ML Libraries         | scikit-learn, scipy, numpy, joblib |
| Encryption Library   | cryptography (Fernet)              |
| Explainability Tools | SHAP                               |
| Data Source          | Kaggle SQL Injection Dataset       |
| IDE / Environment    | Visual Studio Code                 |
| Operating System     | Windows 11 / macOS 10.15.7         |

## IV. RESULTS AND DISCUSSION

### A. GUI Output Evaluation

The installed Tkinter powered graphical interface permits users to enter SQL queries, real time predictions, and inspect encrypted logs. Figures 1, 2, 3, and 4 show four representative outputs being obtained of the system.

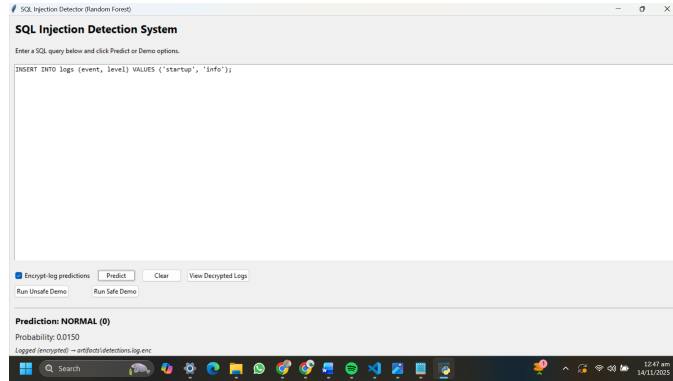


Fig. 1: GUI output of prediction of a normal (benign) SQL query.

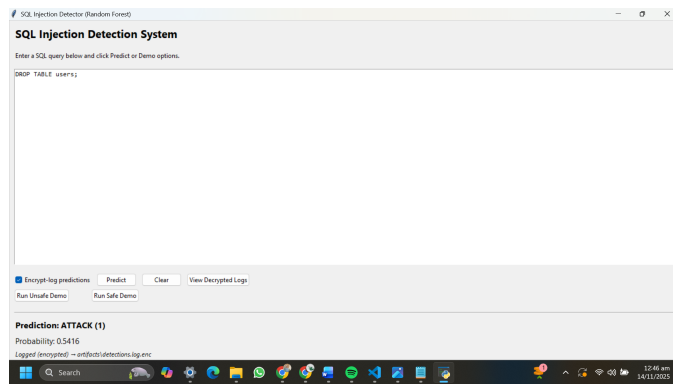


Fig. 2: The output of the GUI indicating the prediction of a malicious SQL injection attack.

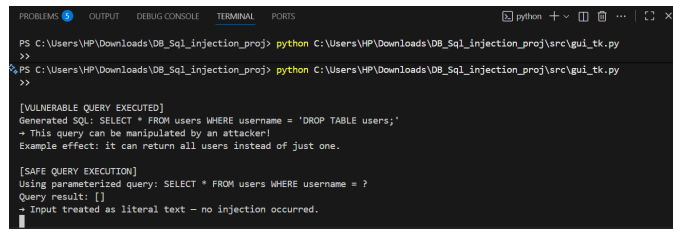


Fig. 3: Output of the Unsafe vs. Safe SQL execution demonstration.

### B. Model Performance Visualization

The random forest (RF) model was trained and tested on SQL queries that they could not see to determine its detection ability. The model is explained by the following figures; analysis of feature importance, ROC curve and confusion matrix.

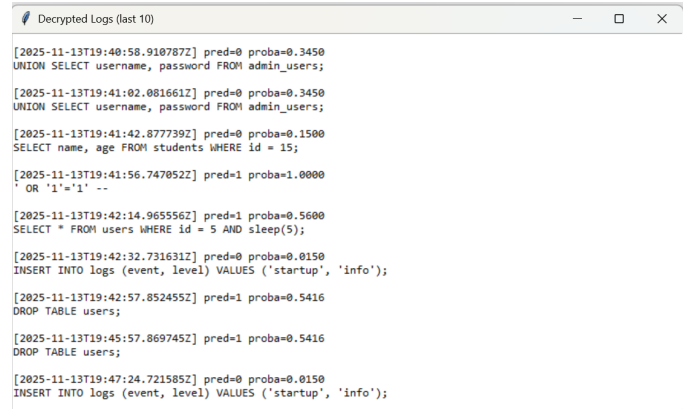


Fig. 4: Decrypted secure logs showing recent detection entries.

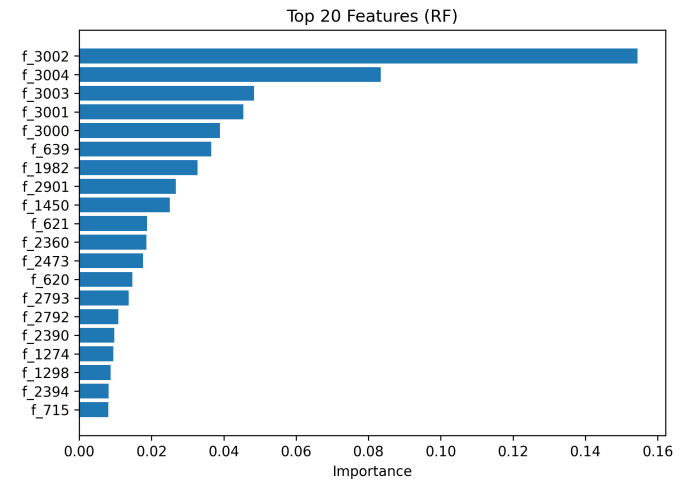


Fig. 5: Top 20 important features identified by the Random Forest model for SQLi detection.

### C. Performance Evaluation

The model uses standard to evaluate itself on unseen data. classification metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Experiment outcomes indicate the Random Forest classifier has a higher level of accuracy and strength than. Logistic Regression and SVM are some of the most common baseline models. Furthermore, it has greater interpretability and reduced variance, ascertaining its suitability in actual practice.

### D. Congruence with Research Objective

The suggested framework is in line with the research objective. of integrating SQL injection detection based on machine learning. and safeguard preventive measures. The hybrid model combining statistically, it is exhibited by the use

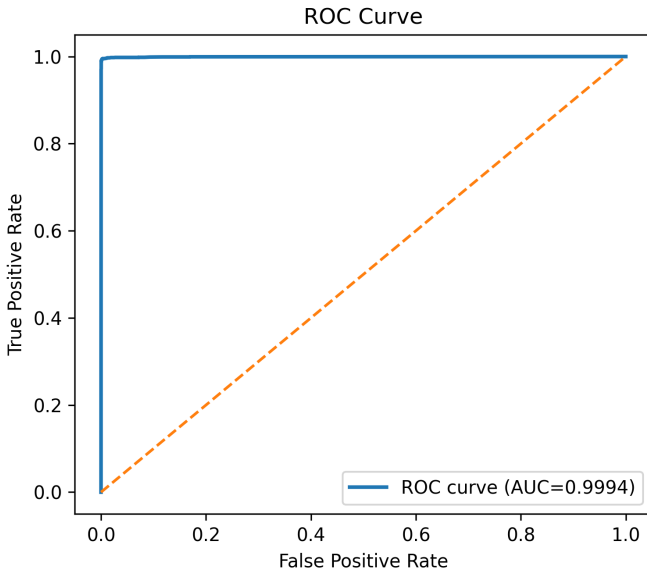


Fig. 6: Receiver Operating Characteristic (ROC) curve showing AUC = 0.9994, indicating near-perfect classification.

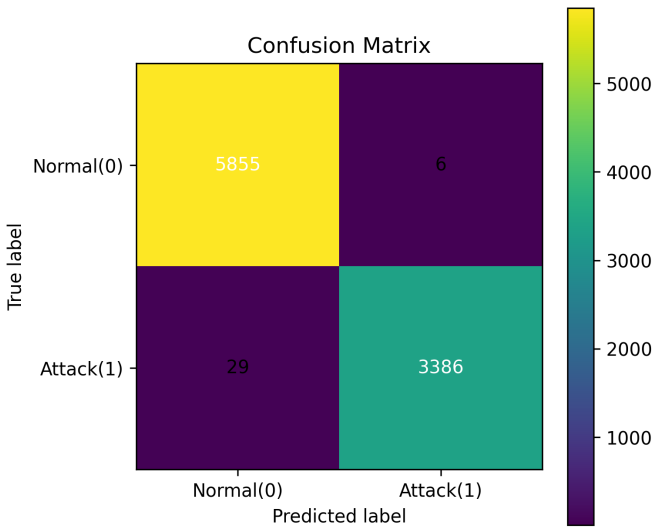


Fig. 7: Confusion matrix displaying the distribution of predictions between normal and malicious SQL queries.

of textual and handcrafted features, large gains in the accuracy of detection.

## V. EVALUATION / COMPARISON

The developed SQL injection detection system with the use of the Random Forests proved to be tested in a quantitative and qualitative manner and was compared to the current methods covered in the literature. Assessment includes model execution, real-time fitness, understandability and feasible security effectiveness.

### A. Comparison of Model Performance

Our Random Forest model was highly accurate and generalized and nearly perfect in discriminating normal and malicious SQL query. Having an AUC = 0.9994, low misclassification, and equal precision-recall accuracy, the model competes with the recent ensemble-based models like the ones by Hussain et al. [4].

In comparison with deep learning methods, including neural network-based SQLi detection [2], Random Forest offers:

- Lower computational cost
- Greater interpretability
- Faster inference time
- More appropriate to real-time detection.

Deep learning models are occasionally more accurate, but the models need a large amount of data preprocessing and additional computational capability. Our Random Forest method, in its turn, offers an effective trade-off between accuracy and efficiency.

Random Forest balances between accuracy, interpretability, and practicality of operations to be used by production-scale systems.

TABLE II: Comparison of Traditional SQL Defense Methods

| Defense Method          | Limitations                                 |
|-------------------------|---------------------------------------------|
| Input validation        | Easily bypassed using obfuscation.          |
| Blacklists / signatures | Fail against zero-day payloads.             |
| Static filtering        | Cannot adapt to new query patterns.         |
| Firewalls (WAFs)        | Struggle with highly obfuscated injections. |

TABLE III: Comparison of ML and DL Techniques

| Technique                           | Strengths                                     | Weaknesses                                          |
|-------------------------------------|-----------------------------------------------|-----------------------------------------------------|
| SVM / Logistic Regression           | Simple, fast                                  | Low accuracy for complex SQL patterns.              |
| Deep Learning (DNNs, LSTM)          | High accuracy, strong feature learning        | High computational cost, low interpretability.      |
| Boosting Models (XGBoost, AdaBoost) | High precision                                | Requires careful tuning; may overfit on small data. |
| <b>Random Forest (Proposed)</b>     | Robust, stable, interpretable, fast inference | Slightly lower accuracy than deep models.           |

### B. Practical System Efficiency

In addition to the accuracy of detection, our system will include:

#### Secure Encrypted Logging

The logs are secure and verifiably confidential using the AES-based Fernet encryption. This is an aspect that is hardly encompassed in the current research prototypes.

#### Real-time GUI Detection System

The Tkinter interface shows:

- Instant predictions
- Visibility of the probability of attack.
- Safe and unsafe query execution.
- Decrypted log viewing

It demonstrates that this framework is not just theoretically good, but it is also practically implementable.

## Prevention Demonstration

The system supports safe development by comparing unsafe execution of SQL statements based on unsafe strings with the use of parameterized queries. This is in line with the modern secure coding provisions and supplements the ML detection element.

### C. Overall Assessment

The proposed system exhibits:

- Very high accuracy and strength.
- More interpretable than deep models.
- Reduced computational demands.
- Anti-virus and anti-tracking services.
- Real-life deployability.

The method, thereby, offers a full pipeline of detection-and-prevention, which enhances earlier studies which emphasized on detection. The hybrid feature extraction strategy and ensemble-based learning enable the system to extrapolate well to the unknown SQL patterns without complicating or obscuring it.

## VI. CONCLUSION

The paper shows that the random forest based hybrid SQL injection detection system can be highly accurate with high generalization and can have practical interpretability, which is confirmed by the previous ensemble-learning research. Key contributions of it are an end-to-end framework that incorporates detection, prevention, encrypted logging, real-time GUI testing, and interpretable machine learning. Although the method is efficient, its weaknesses are its reliance upon a dataset, which lacks the variation in practice, and it may be computationally expensive when large scale live traffic is required. Future developments should be directed to also feature real-time streaming data, go deeper to deep or hybrid ensemble models, and also to incorporate automated SQLi blocking mechanisms. The strengths of the method are its strength, interpretability, efficiency and applicability to a production environment, and the weaknesses include the scale and the diversity of data. The system can assist in the secure web development practices, supplement intrusion detection systems and offer practical tools to monitor, diagnose and prevent SQL injection attacks in the contemporary web infrastructures in real-world applications.

## REFERENCES

- [1] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/Top10/>
- [2] H. Zhang, Y. Lin, and J. Wu, "A Semantic Learning-Based SQL Injection Attack Detection Technology (synBERT)," *\*Electronics\**, vol. 12, no. 4, 2023.
- [3] M. Rahman, F. Hossain, and A. Khan, "SQL Injection Vulnerability Detection Using Deep Learning: A Feature-based Approach," *\*International Journal of Electrical and Computer Engineering (IJECE)\**, vol. 13, no. 2, 2023.
- [4] A. Hussain, M. Liu, and R. Chen, "Enhancing Structured Query Language Injection Detection with Trustworthy Ensemble Learning and Boosting Models Using Local Explanation Techniques," *\*Electronics\**, vol. 13, no. 22, 2023.
- [5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *\*Advances in Neural Information Processing Systems\**, vol. 30, 2017.
- [6] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?" Explaining the Predictions of Any Classifier," *\*Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining\**, 2016, pp. 1135–1144.
- [7] Md. Maruf Hassan and R. Badlishah Ahmad and Tonmoy Ghosh, "SQL Injection Vulnerability Detection Using Deep Learning: A Feature-based Approach" *Indonesian Journal of Electrical Engineering and Informatics (IJEI)*, Vol. 9, No. 3, September 2021, pp. 702 718
- [8] R. Halfond, J. Viegas, and A. Orso, "A classification of SQL injection attacks and countermeasures," in *Proc. IEEE Int. Symp. Secure Softw. Eng.*, 2006.
- [9] R. Valeur, D. Balzarotti, and G. Vigna, "S-QL: A self-learning SQL-injection prevention system," in *Proc. Conf. Detection Intrusions Malware Vulnerability Assessment*, 2005, pp. 85–100.
- [10] S. Wiese, H. Boettcher, and J. Grossklags, "Dynamic taint analysis for detecting SQL injection attacks," in *Proc. IEEE TrustCom*, 2016, pp. 360–367.
- [11] Y. Huang, F. Yu, C. Hang, C. Tsai, D. Lee, and S. Kuo, "Securing web application code by static analysis and runtime protection," in *Proc. Int. Conf. Software Eng.*, 2004, pp. 654–663.
- [12] F. Sun, X. Wang, and C. Zhou, "A machine learning-based SQL injection detection approach," *Journal of Information Security and Applications*, vol. 63, p. 102928, 2021.
- [13] K. Lee, H. Park, and S. Lee, "Deep learning model for SQL injection attack detection," *IEEE Access*, vol. 9, pp. 134392–134402, 2021.
- [14] A. Khare, M. Gupta, and P. Jain, "Hybrid rule-based and machine-learning framework for web-based SQL injection prevention," *Computers & Security*, vol. 112, p. 102510, 2022.
- [15] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Order-preserving encryption for numeric data," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2004, pp. 563–574.
- [16] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, "CryptDB: Protecting confidentiality with encrypted query processing," in *Proc. ACM SOSP*, 2011, pp. 85–100.
- [17] A. Boldyreva, N. Chenette, and A. O'Neill, "Order-preserving encryption revisited: Improved security analysis and alternative solutions," in *Advances in Cryptology – CRYPTO 2011*, 2011, pp. 578–595.
- [18] A. Subashini and V. Kavitha, "A survey on security issues in service delivery models of cloud computing," *Journal of Network and Computer Applications*, vol. 34, no. 1, pp. 1–11, 2011.
- [19] E. Bertino and R. Sandhu, "Database security—concepts, approaches, and challenges," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 1, pp. 2–19, 2005.
- [20] Y. Zhang, H. Xiong, and X. Zhang, "Survey on SQL injection detection and prevention," *Computers & Security*, vol. 105, p. 102246, 2021.
- [21] Sajid. "SQL Injection Dataset." Kaggle. Available: <https://www.kaggle.com/datasets/sajid576/sql-injection-dataset>. Accessed: Feb. 12, 2025.